Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 6

Inside the xv6 File System

Track B: Build Your Own Small OS (xv6)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. Reading What xv6 Shows **2**

2.1 Observe answers 2

3 Part 2. The mystat Program **3**

3.1 Observe answers 3

4 Part 3. Measuring File-System Limits **3**

4.1 Observe answers 4

5 Part 4. The Recursive find Program **4**

5.1 Observe answers 5

6 Conclusion **5**

1. Purpose of the Work

The purpose of this laboratory work is to connect xv6 filesystem concepts with the source code and with observable behavior: file types, directory entries, inodes, hard links, block addressing, filesystem limits, and recursive traversal with a custom `find` program.

2. Part 1. Reading What xv6 Shows

```
$ grep -n "T_DIR\|T_FILE\|T_DEVICE" kernel/stat.h
1:#define T_DIR    1 // Directory
2:#define T_FILE   2 // File
3:#define T_DEVICE 3 // Device
```

Figure 1.1 – Actual xv6 file-type constants

```
$ ls a
.              1 26 32
..             1 1 1024
```

Figure 1.2 – Actual listing of a newly created xv6 directory

```
struct dirent {
  ushort inum;
  char name[DIRSIZ] __attribute__((nonstring));
};
#define DIRSIZ 14
```

Figure 1.3 – Actual directory-entry definition inspected in kernel/fs.h

2.1. Observe answers

1. Type 1 is `T_DIR`, type 2 is `T_FILE`, and type 3 is `T_DEVICE`.
2. The root directory uses inode 1. Both `.` and `..` in the root point to inode 1 because root is its own filesystem parent.
3. An empty directory `a` has size 32 bytes because it contains two 16-byte directory entries, `.` and `..`.

3. Part 2. The mystat Program

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
```

Figure 2.1 – Actual mystat output for a file, directory and device

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
```

Figure 2.2 – Actual hard-link metadata observed with mystat

The inode number assigned to a device can change after `fs.img` is rebuilt with a different set of user programs because `mkfs` recreates the image and assigns inodes again.

3.1. Observe answers

1. After `ln README r2`, both names referred to inode 2 and the link count became 2. Removing `README` left `r2` pointing to the same data while the count returned to 1.
2. `sys_unlink` decrements `ip->nlink`; blocks are reclaimed only after the inode has no directory links and no remaining in-memory reference.
3. The on-disk xv6 inode contains type, major, minor, nlink, size and block addresses. This tree does not store Unix permission bits or a symbolic-link file type.

4. Part 3. Measuring File-System Limits

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define
   DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

Figure 3.1 – Actual xv6 filesystem constants

```
$ make clean
$ make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13,
bitmap blocks 1) blocks 1953 total 2000
```

Figure 3.2 – Actual filesystem layout reported by mkfs

```
$ mkdir averyveryverylongname
$ ls
averyveryveryl 1 28 32
```

Figure 3.3 – Actual truncation of an xv6 filename to DIRSIZ characters

4.1. Observe answers

1. The requested long name appears as `averyveryveryl` because `DIRSIZ` is 14.
2. With 12 direct blocks and 256 indirect entries, the maximum file size is $(12 + 256) \times 1024 = 274432$ bytes, or 268 KiB.
3. The image contained 2000 blocks: 47 metadata blocks and 1953 data blocks in the observed build.

5. Part 4. The Recursive find Program

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
```

```
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
```

Figure 4.1 – Actual recursive find test for r2

```
$ find a b
a/b
```

Figure 4.2 – Actual recursive find test for directory b

5.1. Observe answers

1. The output confirms that `find` traverses nested directories recursively and prints every matching path.
2. The program skips `.` and `..`, preventing an infinite recursion cycle, and ignores directory entries whose inode number is zero.
3. Each directory descriptor is closed after use, which is important because xv6 provides only a small number of open file descriptors per process.

6. Conclusion

The work connected xv6's small filesystem API with its data structures and on-disk limits. Hard-link experiments demonstrated inode sharing, the constants in `fs.h` explained the name and file-size limits, and the custom `find` program used directory entries and recursive traversal to search the filesystem.